



DOCENTE: Thiago Naves

DATA: 01/06/2026

ALUNOS: Eduardo Andrei Staudt e Brandon Monteiro Donisthorpe

RAs: 2783045 e 2758890

TRABALHO - Explorador de Arquivos

1. Objetivo

Este trabalho tem como objetivo desenvolver um programa em C que simula uma linha de comando para manipular diretórios e arquivos usando uma árvore genérica. O programa lê um arquivo in.txt com a lista de pastas e arquivos, monta a árvore na memória e oferece comandos para navegar, buscar, criar e remover itens, do mesmo jeito que um terminal de verdade faz.

2. Ferramentas Utilizadas

- VsCode - para construção do projeto;
- Excalidraw - para desenhos;
- Google Docs - para a escrita da documentação;
- Valgrind - para conferir se não sobrou memória alocada.

3. Estruturas de Dados Utilizadas

enum TipoNo

```
C/C++  
typedef enum {  
    PASTA,
```



```
ARQUIVO  
} TipoNo;
```

Serve para marcar se um nó é uma pasta ou um arquivo. Tem só dois valores, PASTA e ARQUIVO. Isso deixa o código mais claro do que usar um número solto para diferenciar os dois.

struct No

```
C/C++  
typedef struct No {  
    char nome[TAMANHO_NOME];  
    TipoNo tipo;  
    struct No *primeiroFilho;  
    struct No *proximoIrmao;  
    struct No *pai;  
} No;
```

Representa um item da árvore, que pode ser uma pasta ou um arquivo. Cada nó guarda o nome, o tipo e três ponteiros. O primeiroFilho aponta para o primeiro item dentro da pasta. O proximoIrmao aponta para o próximo item que está no mesmo nível. O pai aponta para a pasta que contém o nó, o que facilita subir um nível e montar o caminho completo.

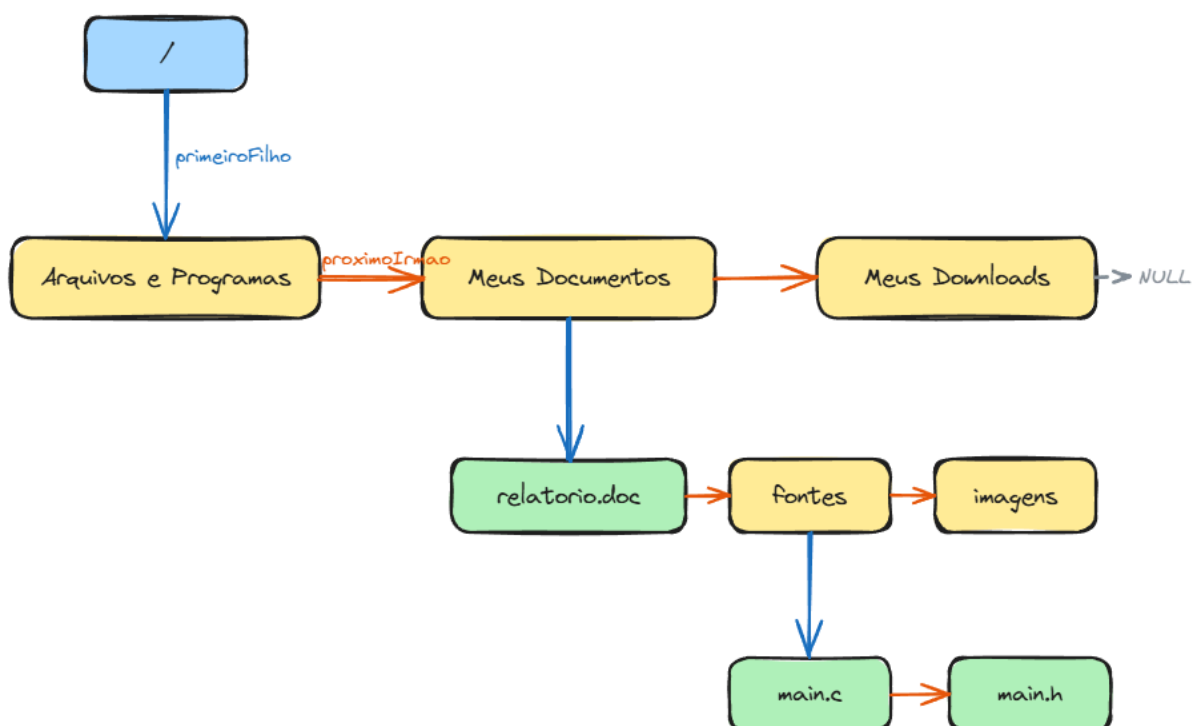
Defines

```
C/C++  
#define TAMANHO_NOME 256  
#define TAMANHO_LINHA 1024
```



TAMANHO_NOME guarda o tamanho máximo do nome de um item. TAMANHO_LINHA guarda o tamanho máximo de uma linha lida do arquivo ou digitada pelo usuário. Usar define deixa esses limites em um lugar só, fácil de mudar depois.

A Árvore Implementada



Imagens 1: representação do formato de árvore utilizado no trabalho. Fonte: Autoria própria prints do terminal.

A árvore usada aqui é uma árvore genérica, ou seja, cada pasta pode ter qualquer quantidade de filhos. Para representar isso sem precisar de um vetor de tamanho fixo, foi usado o modelo filho-irmão.

A ideia é simples. Cada nó aponta para apenas um filho, o primeiro, através do primeiroFilho. Os outros itens daquela mesma pasta ficam ligados em sequência pelo



proximolrmao, formando uma lista. Então para listar tudo que tem dentro de uma pasta, o programa pega o primeiroFilho e vai seguindo os proximolrmao até chegar em NULL.

O ponteiro pai existe para o caminho contrário. Com ele dá para subir de uma pasta para a que está acima, o que é usado no comando `cd ..` e na hora de montar o caminho completo de um item. A raiz é um nó especial com o nome `/` e sem pai, e é dela que toda navegação começa.

Leitura do Arquivo in.txt

```
C/C++  
Arquivos e Programas/Firefox  
Arquivos e Programas/Chrome  
Arquivos e Programas/Opera  
Meus Documentos/apresentacao.ppt  
Meus Documentos/relatorio.doc  
Meus Documentos/fontes  
Meus Documentos/fontes/main.c  
Meus Documentos/fontes/main.h  
Meus Documentos/imagens  
Meus Downloads/7zip.exe  
Meus Downloads/t2.rar
```

O arquivo in.txt guarda um caminho por linha, com as pastas separadas por barra. Um exemplo é Meus Documentos/fontes/main.c, que quer dizer que main.c está dentro de fontes, que está dentro de Meus Documentos.

A função de leitura abre o arquivo e lê linha por linha. Para cada linha, ela tira o caractere de quebra de linha do final e quebra o texto nos pontos onde tem barra. Aí ela caminha pela árvore a partir da raiz. Para cada parte do caminho, ela verifica se já existe uma pasta com aquele nome naquele nível. Se já existe, ela só entra nela e reaproveita. Se não existe, cria um nó novo. Por isso uma pasta que aparece em várias linhas, como fontes, é criada uma vez só.

Para saber se um item é pasta ou arquivo, o programa olha se o nome tem um ponto. Se tiver ponto, é arquivo, porque o arquivo sempre tem extensão como .c ou .doc. Se não tiver, é pasta.



4. Funções cmd.c

Função criarNo

```
C/C++
No* criarNo(char nome[], TipoNo tipo) {
    No *novo = (No*) malloc(sizeof(No));
    if (novo == NULL) {
        printf("Erro de alocação de memória\n");
        exit(1);
    }
    strcpy(novo->nome, nome);
    novo->tipo = tipo;
    novo->primeiroFilho = NULL;
    novo->proximoIrmao = NULL;
    novo->pai = NULL;
    return novo;
}
```

A função reserva memória para um nó novo, copia o nome recebido, guarda o tipo e deixa os três ponteiros apontando para NULL. Se a alocação falhar, mostra uma mensagem e encerra o programa. Se der certo, retorna o ponteiro para o nó criado.

Função criarRaiz

```
C/C++
No* criarRaiz() {
    No *raiz = criarNo("/", PASTA);
    return raiz;
}
```



A função cria o nó raiz da árvore. Ela chama criarNo com o nome / e o tipo pasta, e retorna esse nó. É a partir da raiz que todo o resto da árvore é montado.

Função descobrirTipo

```
C/C++
TipoNo descobrirTipo(char nome[]) {
    int i;
    for (i = 0; nome[i] != '\0'; i++) {
        if (nome[i] == '.') {
            return ARQUIVO;
        }
    }
    return PASTA;
}
```

A função recebe um nome e percorre os caracteres procurando um ponto. Se encontrar um ponto, retorna ARQUIVO. Se chegar ao fim sem achar nenhum, retorna PASTA. É assim que o programa separa arquivos de pastas na hora de montar a árvore.

Função buscarFilho

```
C/C++
No* buscarFilho(No *pasta, char nome[]) {
    No *filho = pasta->primeiroFilho;
    while (filho != NULL) {
        if (strcmp(filho->nome, nome) == 0) {
            return filho;
        }
        filho = filho->proximoIrmao;
    }
    return NULL;
}
```



```
}
```

A função procura um filho direto de uma pasta pelo nome. Ela começa no primeiroFilho e vai seguindo os proximoIrmão, comparando o nome de cada um. Se achar, retorna o ponteiro para esse filho. Se percorrer todos sem achar, retorna NULL.

Função adicionarFilho

```
C/C++  
void adicionarFilho(No *pai, No *filho) {  
    filho->pai = pai;  
    if (pai->primeiroFilho == NULL) {  
        pai->primeiroFilho = filho;  
        return;  
    }  
    No *atual = pai->primeiroFilho;  
    while (atual->proximoIrmão != NULL) {  
        atual = atual->proximoIrmão;  
    }  
    atual->proximoIrmão = filho;  
}
```

A função liga um nó novo como filho de uma pasta. Primeiro ela marca o pai do novo nó. Se a pasta ainda não tem nenhum filho, o novo vira o primeiroFilho. Se já tem, a função caminha pela lista de irmãos até o último e prende o novo no final, para manter a ordem em que os itens foram inseridos.



Função inserirCaminho

```
C/C++  
No* inserirCaminho(No *raiz, char caminho[]) {  
    No *atual = raiz;  
    char copia[TAMANHO_LINHA];  
    strcpy(copia, caminho);  
  
    char *parte = strtok(copia, "/");  
    while (parte != NULL) {  
        No *filho = buscarFilho(atual, parte);  
        if (filho == NULL) {  
            filho = criarNo(parte, descobrirTipo(parte));  
            adicionarFilho(atual, filho);  
        }  
        atual = filho;  
        parte = strtok(NULL, "/");  
    }  
    return atual;  
}
```

A função liga um nó novo como filho de uma pasta. Primeiro ela marca o pai do novo nó. Se a pasta ainda não tem nenhum filho, o novo vira o primeiroFilho. Se já tem, a função caminha pela lista de irmãos até o último e prende o novo no final, para manter a ordem em que os itens foram inseridos.

Função lerArquivo

```
C/C++  
void lerArquivo(No *raiz, char nomeArquivo[]) {  
    FILE *arquivo = fopen(nomeArquivo, "r");  
    if (arquivo == NULL) {
```



```
printf("Nao foi possivel abrir o arquivo %s\n", nomeArquivo);  
return;  
}  
  
char linha[TAMANHO_LINHA];  
while (fgets(linha, TAMANHO_LINHA, arquivo) != NULL) {  
    int tamanho = strlen(linha);  
    while (tamanho > 0 && (linha[tamanho - 1] == '\n' || linha[tamanho - 1]  
== '\r')) {  
        linha[tamanho - 1] = '\0';  
        tamanho--;  
    }  
    if (tamanho > 0) {  
        inserirCaminho(raiz, linha);  
    }  
}  
fclose(arquivo);  
}
```

A função abre o in.txt e lê linha por linha. Se o arquivo não abrir, mostra um aviso e retorna sem quebrar o programa. Para cada linha, tira a quebra de linha do final e, se a linha não estiver vazia, chama `inserirCaminho` para montar aquele caminho na árvore. No fim fecha o arquivo.

Função montarCaminho

```
C/C++  
void montarCaminho(No *no, char destino[]) {  
    if (no->pai == NULL) {  
        strcpy(destino, "/");  
    }  
}
```



```
        return;
    }
    montarCaminho(no->pai, destino);
    if (strcmp(destino, "/") != 0) {
        strcat(destino, "/");
    }
    strcat(destino, no->nome);
}
```

A função monta o caminho completo de um nó, da raiz até ele. Ela funciona de forma recursiva, subindo pelo ponteiro pai até chegar na raiz, e vai juntando os nomes com barra na volta. O resultado é uma string como /Meus Documentos/fontes, usada no comando search e no prompt do terminal.

Função comandoCd

```
C/C++
void comandoCd(No **atual, char argumento[]) {
    if (strcmp(argumento, "..") == 0) {
        if ((*atual)->pai != NULL) {
            *atual = (*atual)->pai;
        }
        return;
    }

    No *destino = buscarFilho(*atual, argumento);

    if (destino != NULL && destino->tipo == PASTA) {
        *atual = destino;
        return;
    }
}
```



```
}  
if (destino != NULL && destino->tipo == ARQUIVO) {  
    printf("%s e um arquivo, nao uma pasta\n", argumento);  
    return;  
}  
  
int encontrouAlternativa = 0;  
int tamanhoArgumento = strlen(argumento);  
No *filho = (*atual)->primeiroFilho;  
while (filho != NULL) {  
    if (filho->tipo == PASTA && strncmp(filho->nome, argumento,  
tamanhoArgumento) == 0) {  
        if (encontrouAlternativa == 0) {  
            printf("Diretorio nao encontrado. Voce quis dizer:\n");  
        }  
        printf("  %s\n", filho->nome);  
        encontrouAlternativa = 1;  
    }  
    filho = filho->proximoIrmao;  
}  
  
if (encontrouAlternativa == 0) {  
    printf("Diretorio nao encontrado\n");  
}  
}
```

A função troca a pasta atual. Se o argumento for cd .., ela sobe para a pasta pai, quando existe. Se for um nome, ela procura uma pasta filho com aquele nome e entra nela. Se o nome bater com um arquivo, avisa que é arquivo e não pasta. Se não achar nada, ela mostra as pastas daquele nível que começam com o texto digitado, como sugestão. Se nem isso existir, mostra Diretório não encontrado.



Função comandoMkdir

C/C++

```
void comandoMkdir(No *atual, char argumento[]) {  
    if (strlen(argumento) == 0) {  
        printf("Informe o nome da pasta. Uso: mkdir <nome>\n");  
        return;  
    }  
    No *existente = buscarFilho(atual, argumento);  
    if (existente != NULL) {  
        printf("Ja existe um item chamado %s nesta pasta\n", argumento);  
        return;  
    }  
    No *nova = criarNo(argumento, PASTA);  
    adicionarFilho(atual, nova);  
    printf("Pasta %s criada\n", argumento);  
}
```

A função cria uma pasta nova dentro da pasta atual. Se o nome vier vazio, mostra como usar o comando. Se já existir um item com aquele nome, avisa e não cria de novo. Se estiver tudo certo, cria o nó como pasta e liga na pasta atual.

Função buscarRecurso

C/C++

```
void buscarRecurso(No *atual, char alvo[], int *contador) {  
    No *filho = atual->primeiroFilho;  
    while (filho != NULL) {  
        if (strcmp(filho->nome, alvo) == 0) {  
            char caminho[TAMANHO_LINHA];  
            montarCaminho(filho, caminho);  
            printf("  %s\n", caminho);  
        }  
        filho = filho->proximoFilho;  
    }  
}
```



```
        *contador = *contador + 1;
    }
    buscarRecursivo(filho, alvo, contador);
    filho = filho->proximoIrmao;
}
}
```

A função procura um nome na árvore inteira de forma recursiva. Ela percorre os filhos de um nó e, para cada um, compara o nome com o que foi buscado. Se bater, monta o caminho completo, imprime e soma um no contador. Depois desce para os filhos daquele nó e repete. É o que permite o search achar itens em qualquer profundidade.

Função comandoSearch

```
C/C++
void comandoSearch(No *raiz, char argumento[]) {
    if (strlen(argumento) == 0) {
        printf("Informe o que buscar. Uso: search <nome>\n");
        return;
    }
    int contador = 0;
    printf("Resultados para \"%s\":\n", argumento);
    buscarRecursivo(raiz, argumento, &contador);
    if (contador == 0) {
        printf("  Nenhum arquivo ou pasta encontrado\n");
    }
}
```



A função inicia a busca a partir da raiz usando `buscarRecursoivo`. Se o argumento vier vazio, mostra como usar o comando. Ela usa um contador para saber quantas vezes achou o nome. Se o contador terminar em zero, avisa que não encontrou nada.

Função `comandoList`

C/C++

```
void comandoList(No *atual) {  
    No *filho = atual->primeiroFilho;  
    if (filho == NULL) {  
        printf("Pasta vazia\n");  
        return;  
    }  
    while (filho != NULL) {  
        if (filho->tipo == PASTA) {  
            printf("[PASTA]   %s\n", filho->nome);  
        } else {  
            printf("[ARQUIVO] %s\n", filho->nome);  
        }  
        filho = filho->proximoIrmao;  
    }  
}
```

A função mostra tudo que está dentro da pasta atual. Ela percorre os filhos pelo `proximoIrmao` e imprime cada um marcando se é pasta ou arquivo. Se a pasta estiver vazia, avisa que está vazia.



Função comandoRm:

C/C++

```
void comandoRm(No *atual, char argumento[]) {  
    if (strlen(argumento) == 0) {  
        printf("Informe o que remover. Uso: rm <nome>\n");  
        return;  
    }  
    No *alvo = buscarFilho(atual, argumento);  
    if (alvo == NULL) {  
        printf("Item %s nao encontrado nesta pasta\n", argumento);  
        return;  
    }  
  
    if (atual->primeiroFilho == alvo) {  
        atual->primeiroFilho = alvo->proximoIrmao;  
    } else {  
        No *anterior = atual->primeiroFilho;  
        while (anterior->proximoIrmao != alvo) {  
            anterior = anterior->proximoIrmao;  
        }  
        anterior->proximoIrmao = alvo->proximoIrmao;  
    }  
  
    alvo->proximoIrmao = NULL;  
    liberarArvore(alvo);  
    printf("%s removido\n", argumento);  
}
```

Se a matriz for NULL, retorna 1. Caso contrário, percorre linha por linha: dir para avançar nas colunas e baixo para descer para a próxima linha. Os valores são impressos com %5d para manter o alinhamento em colunas. Retorna 0 ao final.



Função comandoClear:

```
C/C++  
void comandoClear() {  
    #ifdef _WIN32  
        system("cls");  
    #else  
        system("clear");  
    #endif  
}
```

A função limpa a tela do terminal. Ela usa `cls` no Windows e `clear` nos outros sistemas, escolhido na hora de compilar com uma diretiva de pré-processador.

Função comandoHelp:

```
C/C++  
void comandoHelp() {  
    printf("Comandos disponiveis:\n\n");  
    printf("cd <pasta>      Entra na pasta indicada. Use cd .. para voltar uma  
pasta.\n");  
    printf("                Se a pasta nao existir, mostra as que comecam  
igual.\n");  
    printf("search <nome> Procura um arquivo ou pasta na arvore inteira e  
mostra o caminho.\n");  
    printf("rm <nome>      Remove uma pasta ou arquivo da pasta atual liberando  
tudo dentro.\n");  
    printf("list          Lista as pastas e arquivos da pasta atual.\n");  
    printf("mkdir <nome>   Cria uma pasta nova dentro da pasta atual.\n");  
    printf("clear        Limpa a tela.\n");  
    printf("help         Mostra esta lista de comandos.\n");  
}
```



```
printf("exit          Libera a memoria e fecha o programa.\n");  
}
```

A função mostra a lista de todos os comandos do programa, explicando para que cada um serve e como usar. É o comando que o usuário chama quando não lembra o que o programa faz.

Função liberarArvore:

```
C/C++  
void liberarArvore(No *no) {  
    if (no == NULL) {  
        return;  
    }  
    No *filho = no->primeiroFilho;  
    while (filho != NULL) {  
        No *proximo = filho->proximoIrmao;  
        liberarArvore(filho);  
        filho = proximo;  
    }  
    free(no);  
}
```

A função libera um nó e tudo que está abaixo dele, de forma recursiva. Para cada filho, ela guarda o próximo irmão antes de liberar, desce e libera aquele filho inteiro, e depois segue para o próximo. No fim libera o próprio nó. É usada tanto no rm quanto no encerramento do programa.

5. Arquivo main.c



C/C++

```
int main() {
    No *raiz = criarRaiz();
    lerArquivo(raiz, "in.txt");

    No *atual = raiz;
    char linha[TAMANHO_LINHA];
    char comando[TAMANHO_LINHA];
    char argumento[TAMANHO_LINHA];
    int rodando = 1;

    printf("Simulador de diretorios. Digite help para ver os comandos.\n");

    do {
        char caminho[TAMANHO_LINHA];
        montarCaminho(atual, caminho);
        printf("\n%s> ", caminho);

        if (fgets(linha, TAMANHO_LINHA, stdin) == NULL) {
            break;
        }

        int tamanho = strlen(linha);
        while (tamanho > 0 && (linha[tamanho - 1] == '\n' || linha[tamanho - 1]
== '\r')) {
            linha[tamanho - 1] = '\0';
            tamanho--;
        }

        int i = 0;
        int j = 0;
        while (linha[i] != '\0' && linha[i] != ' ') {
            comando[j] = linha[i];
            i++;
            j++;
        }
        comando[j] = '\0';

        while (linha[i] == ' ') {
            i++;
        }
    }
```



```
j = 0;
while (linha[i] != '\0') {
    argumento[j] = linha[i];
    i++;
    j++;
}
argumento[j] = '\0';

if (strcmp(comando, "cd") == 0) {
    comandoCd(&atual, argumento);
} else if (strcmp(comando, "search") == 0) {
    comandoSearch(raiz, argumento);
} else if (strcmp(comando, "rm") == 0) {
    comandoRm(atual, argumento);
} else if (strcmp(comando, "list") == 0) {
    comandoList(atual);
} else if (strcmp(comando, "mkdir") == 0) {
    comandoMkdir(atual, argumento);
} else if (strcmp(comando, "clear") == 0) {
    comandoClear();
} else if (strcmp(comando, "help") == 0) {
    comandoHelp();
} else if (strcmp(comando, "exit") == 0) {
    rodando = 0;
} else if (strcmp(comando, "") == 0) {
    rodando = 1;
} else {
    printf("Comando desconhecido: %s. Digite help.\n", comando);
}

} while (rodando == 1);

liberarArvore(raiz);
printf("Memoria liberada. Programa encerrado.\n");
return 0;
}
```

O main.c é onde o programa começa. Primeiro ele cria a raiz e chama lerArquivo para montar a árvore com o conteúdo do in.txt. Depois entra em um laço do while que só termina quando o usuário digita exit.



Dentro do laço, o programa mostra o caminho da pasta atual como prompt e lê a linha inteira que o usuário digitou. Ele separa a primeira palavra, que é o comando, do resto, que é o argumento. Essa separação manual mantém os espaços do argumento, então nomes com espaço como Meus Documentos funcionam normalmente.

Com o comando em mãos, uma sequência de comparações decide qual função chamar. Quando o usuário digita exit, o laço para, a árvore inteira é liberada com liberarArvore e o programa encerra.

Desafios e Dificuldades

O maior desafio foi entender o modelo filho-irmão. No começo a ideia de cada nó ter só um ponteiro para filho e outro para irmão confunde, porque dá a impressão de que a pasta só pode ter um item dentro. Depois ficou claro que os outros itens ficam ligados em sequência pelo irmão. Para o modelo filho-irmão, desenhar a árvore no papel e no Excalidraw ajudou a enxergar como os ponteiros se ligam e a escrever as funções de percorrer e listar com mais segurança.

Outra dificuldade foi a liberação de memória no rm. Como os nós ficam ligados por ponteiros, na hora de apagar uma pasta era fácil acabar apagando os irmãos dela junto por engano, ou perder o ponteiro para o resto da lista. Foi preciso pensar bem em qual ponteiro salvar antes de chamar o free. Para a liberação no rm, a solução foi guardar o próximo irmão em uma variável antes de liberar o nó atual, e fazer a função de liberar trabalhar só com os filhos do nó, sem mexer nos irmãos dele. Assim o rm desliga o item da lista primeiro e só depois libera ele e o que estava dentro.

Também deu trabalho separar o comando do argumento mantendo os espaços. Os nomes de pasta do trabalho têm espaço, como Meus Documentos, então não dava para simplesmente quebrar o texto em todo espaço. Foi preciso pegar só a primeira palavra como comando e deixar todo o resto como argumento. Para a separação de comando e argumento, o programa caminha pela linha digitada copiando os caracteres até o primeiro espaço para formar o comando, pula os espaços do meio e copia todo o resto para o argumento. Com isso os nomes com espaço continuam inteiros.



Comandos implementados

cd <pasta>

Entra na pasta indicada. Com cd .. volta uma pasta. Se a pasta não existir, mostra as pastas do nível atual que começam com o texto digitado.

```
/> cd Meus Documentos  
  
/Meus Documentos> |
```

search <nome>

Procura um arquivo ou pasta na árvore inteira e mostra o caminho completo de cada ocorrência.

```
/Meus Documentos> search imagens  
Resultados para "imagens":  
/Meus Documentos/imagens
```

rm <nome>

Remove uma pasta ou arquivo da pasta atual. Se for pasta, apaga também tudo que estava dentro.

```
/Meus Documentos> rm qualquercoisa  
qualquercoisa removido  
  
/Meus Documentos> list  
[ARQUIVO] apresentacao.ppt  
[ARQUIVO] relatorio.doc  
[PASTA] fontes  
[PASTA] imagens
```

list

Lista as pastas e arquivos que estão dentro da pasta atual.

```
/> list  
[PASTA] Arquivos e Programas  
[PASTA] Meus Documentos  
[PASTA] Meus Downloads
```



mkdir <nome>

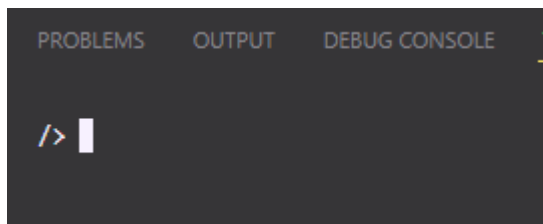
Cria uma pasta nova dentro da pasta atual.

```
/Meus Documentos> mkdir qualquercoisa
Pasta qualquercoisa criada

/Meus Documentos> list
[ARQUIVO] apresentacao.ppt
[ARQUIVO] relatorio.doc
[PASTA] fontes
[PASTA] imagens
[PASTA] qualquercoisa
```

clear

Limpa a tela do terminal.



help

Mostra todos os comandos, o modo de usar e a finalidade de cada um.

```
/> help
Comandos disponiveis:

cd <pasta>      Entra na pasta indicada. Use cd .. para voltar uma pasta.
                Se a pasta nao existir, mostra as que comecam igual.
search <nome>   Procura um arquivo ou pasta na arvore inteira e mostra o caminho.
rm <nome>       Remove uma pasta ou arquivo da pasta atual liberando tudo dentro.
list           Lista as pastas e arquivos da pasta atual.
mkdir <nome>    Cria uma pasta nova dentro da pasta atual.
clear          Limpa a tela.
help           Mostra esta lista de comandos.
exit           Libera a memoria e fecha o programa.

/>
```



exit

Libera toda a memória usada e fecha o programa.

```
/> exit  
Memoria liberada. Programa encerrado.  
PS C:\dev\projects\AEDIII\Trabalhos\T2-Árvores> |
```

Gerenciamento de Memória

Toda a memória da árvore é alocada de forma dinâmica com malloc, na função criarNo, uma vez para cada pasta ou arquivo. Conforme o usuário cria pastas com mkdir ou o arquivo é lido, novos nós vão sendo alocados.

A liberação acontece em dois momentos. No comando rm, quando o usuário apaga uma pasta, a função liberarArvore libera de forma recursiva o nó e todos os que estavam dentro dele, sem deixar nenhum solto. No comando exit, a mesma função é chamada na raiz, o que libera a árvore inteira de uma vez antes do programa fechar.

Para conferir que não ficou memória sobrando, o programa foi testado com o Valgrind, que acusou zero bytes em uso no final e nenhum erro.

```
==532== HEAP SUMMARY:  
==532==    in use at exit: 0 bytes in 0 blocks  
==532== total heap usage: 20 allocs, 20 frees, 17,368 bytes allocated  
==532==  
==532== All heap blocks were freed -- no leaks are possible  
==532==  
==532== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Como Compilar e Executar

O programa é compilado com o gcc usando os três arquivos. No Linux ou Mac o comando é:



```
C/C++  
gcc -Wall cmd.c main.c -o programa  
./programa
```

No Windows:

```
C/C++  
gcc cmd.c main.c -o programa.exe  
./programa.exe
```

O arquivo in.txt precisa estar na mesma pasta do executável, porque o programa lê ele assim que abre. Ao rodar, o programa monta a árvore e mostra o prompt esperando os comandos.

Conclusão

O objetivo do trabalho foi cumprido. O programa monta uma árvore genérica de diretórios a partir de um arquivo, e deixa o usuário navegar e mexer nessa árvore por uma linha de comando parecida com a de um terminal de verdade.

Todos os comandos pedidos foram implementados e testados, cd, search, rm, list, mkdir, clear, help e exit. A escolha do modelo filho-irmão permitiu que cada pasta tivesse qualquer quantidade de itens dentro, sem limite fixo. A parte que mais deu trabalho foi a liberação de memória, que precisa ser recursiva para apagar uma pasta e tudo que está dentro dela sem deixar nada solto.

O trabalho ajudou a fixar os conteúdos de árvores, ponteiros e alocação dinâmica vistos em aula, e mostrou na prática como uma estrutura de dados encadeada pode representar algo do dia a dia como pastas e arquivos.